Introduzione a C++

Indice

Introduzione

Caratteristiche principali del linguaggio di programmazione

Variabili e tipi di dato

Flusso di ingresso e uscita

Operatori

Istruzioni condizionali

Cicli

Array

La funzione `main` e le funzioni definite dall'utente

Un algoritmo completo: il valore massimo in un array

Operatori bit a bit

Punti chiave del capitolo

Introduzione

Introduzione: dallo pseudocodice a C++

- Nel capitolo «Pseudocodice» gli algoritmi sono scritti in un linguaggio pensato per le persone
- È preciso, ma **non viene eseguito** da un calcolatore
- Per eseguire un algoritmo dobbiamo tradurlo in un **linguaggio di programmazione**
- In questo capitolo introduciamo il linguaggio di programmazione C++, che usiamo per implementare gli algoritmi del corso

Il percorso del capitolo

Il capitolo segue lo stesso percorso del capitolo «Pseudocodice»:

- le caratteristiche principali del linguaggio e la struttura di un programma
- variabili, tipi di dato e flusso di ingresso e uscita
- operatori, istruzioni condizionali e cicli
- gli array, la cui dimensione può essere fissata o decisa durante l'esecuzione
- le funzioni, con la funzione `main` come punto di ingresso di ogni programma
- un algoritmo completo, la ricerca del valore massimo in un array, scritto in pseudocodice e in `C++`
- gli operatori bit a bit, che agiscono direttamente sulla rappresentazione binaria dei numeri interi

Ogni frammento di codice è stato eseguito

Ogni frammento di codice del capitolo
è stato compilato con il compilatore g++ (standard C++17)
ed eseguito.

Gli output riportati negli esempi
sono quelli stampati dai programmi.

Indici: pseudocodice e C++

- Nello pseudocodice le componenti di un array $a[1..n]$ sono indicizzate da 1 a n
- In C++ le componenti di un array di dimensione n sono indicizzate da 0 a $n-1$
- La componente $a[i]$ dello pseudocodice corrisponde alla componente `a[i-1]` del codice C++

Riepilogo

- Lo pseudocodice è preciso ma non viene eseguito: per eseguire un algoritmo lo scriviamo in un linguaggio di programmazione, qui `C++`
- Il capitolo segue il percorso del capitolo «Pseudocodice», fino a un algoritmo completo e agli operatori bit a bit
- Gli indici dello pseudocodice vanno da 1 a n, quelli di `C++` da 0 a $n - 1$

Caratteristiche principali del linguaggio di programmazione

Introduzione: caratteristiche principali di C++

- Descriviamo prima che tipo di linguaggio è C++
- Poi osserviamo lo scheletro che ogni programma C++ ha in comune, e come un programma viene compilato ed eseguito
- È la cornice in cui sono scritte tutte le istruzioni delle sezioni seguenti

Caratteristiche principali (1/2)

- C++ è un linguaggio **imperativo** che dà accesso diretto alla memoria e alle risorse hardware del calcolatore
- È molto efficiente per la programmazione di sistema a basso livello e allo stesso tempo supporta astrazioni di alto livello
- Questo lo rende versatile sia per le applicazioni in cui le prestazioni sono critiche sia per lo sviluppo software moderno
- C++ supporta le **funzioni**, che permettono un approccio strutturato e modulare alla programmazione
- Le funzioni consentono di riutilizzare il codice e di organizzarlo meglio

Caratteristiche principali (2/2)

- C++ comprende un insieme predefinito di **simboli riservati** usati per dichiarare variabili, definire funzioni e specificare le istruzioni eseguibili
- Il **codice sorgente** è costituito da una sequenza di **operazioni atomiche** eseguite in ordine, una dopo l'altra, su variabili di **tipi di dato** specifici
- Il codice sorgente può contenere **commenti**, che servono come documentazione del codice e vengono ignorati dal compilatore
- Un commento su una sola riga inizia con il simbolo riservato "//"
- I commenti su più righe sono racchiusi fra i simboli riservati "/*" e "*/"

Esempio: commenti in C++

Questo è un esempio di commento su una sola riga:

```
1 // Questa è una riga di commento
```

Questo è un esempio di commento su più righe:

```
1 /* This is a
2    multi-line comment */
```

La struttura di un programma

- Prima di descrivere le singole istruzioni, osserviamo lo scheletro che ogni programma ha in comune
- Un programma `C++` è un file di testo (con estensione `.cpp`) che viene tradotto da un **compilatore** in un file eseguibile
- L'esecuzione del programma parte dalla funzione `main`, il punto di ingresso di ogni programma

Esempio: lo scheletro di un programma (1/2)

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      // Qui si scrivono le istruzioni del programma
8      cout << "Ciao, mondo!" << endl;
9
10     return 0;
11 }
```

Esempio: lo scheletro di un programma (2/2)

Se il codice sorgente è salvato nel file `hello.cpp`, lo si compila e lo si esegue con i comandi

```
g++ -Wall -std=c++17 -o hello hello.cpp
./hello
```

e l'output è

```
Ciao, mondo!
```

Frammenti di codice

Negli esempi seguenti scriviamo
solo le istruzioni che contano.

A meno che non contengano un programma completo,
i frammenti di codice vanno inseriti
fra le parentesi graffe di `main`.

Ogni istruzione termina con un punto e virgola

Ogni istruzione di un codice sorgente C++
deve terminare con il simbolo riservato “;”.

Riepilogo

Il linguaggio:

- `C++` è imperativo, efficiente, e supporta le funzioni
- Il codice sorgente è una sequenza di operazioni atomiche su variabili di tipi di dato specifici
- Commenti: "`//`" per una riga, "`/* ... */`" per più righe

La struttura di un programma:

- Un file `.cpp` viene compilato (ad esempio con `g++`) in un file eseguibile
- L'esecuzione parte da `main`; ogni istruzione termina con "`;`"

Variabili e tipi di dato

Introduzione: variabili e tipi di dato

- Il codice sorgente opera su **variabili**, e ogni variabile ha un **tipo di dato**
- Vediamo i tipi di dato fondamentali di `C++` e come si dichiarano e si assegnano le variabili
- Ogni algoritmo del corso memorizza i suoi dati in variabili di questi tipi

Tipi di dato (1/2)

Le informazioni sono classificate in diversi **tipi di dato**. Le quattro famiglie fondamentali di tipi di dato sono:

1. Il tipo di dato "`int`" rappresenta un numero intero, cioè un numero scritto senza la virgola decimale. Comprende i numeri interi positivi e negativi, e lo zero.

2. Il tipo di dato "`bool`" rappresenta un valore booleano: `true` o `false`.

Tipi di dato (2/2)

3. Il tipo di dato "`float`" rappresenta un numero in virgola mobile, che comprende i numeri scritti con la virgola decimale. Il tipo di dato "`double`" rappresenta un numero in virgola mobile a precisione doppia, che offre maggiore accuratezza e un intervallo più ampio di `float`.

4. Il tipo di dato "`char`" rappresenta un singolo carattere, come una lettera, una cifra o un simbolo, racchiuso fra apici singoli (ad esempio `'A'`, `'7'`, `'!'`).

- Una sequenza di caratteri racchiusa fra virgolette doppie (ad esempio `"Ada"`) è memorizzata in una variabile di tipo "`string`", fornito dalla libreria standard `<string>`

Variabili

- Una variabile funziona come un contenitore che memorizza un valore di un tipo di dato specifico per un uso successivo
- Ogni variabile ha un nome univoco e contiene un solo valore in ogni istante
- Alle variabili si assegnano valori con l'operatore di assegnamento "="
- Quando si assegna un nuovo valore, il valore precedente viene sovrascritto

Dichiarazione prima dell'uso

Le variabili devono essere dichiarate prima dell'uso, specificandone il tipo di dato.

Esempio: dichiarazioni di variabili e assegnamenti di valori (1/2)

```
1  int age = 25; // Variabile intera
2  bool isRaining = false; // Variabile booleana
3  float smallNumber = 3.14; // Variabile in virgola
       mobile
4  double largeNumber = 123456.789; // Variabile in
       virgola mobile a doppia precisione
5  char character = 'a'; // Variabile carattere
6  string name = "Ada"; // Variabile stringa
7
8  age = 26; // Il valore precedente 25 viene sovrascritto
9  cout << age << " " << isRaining << " " << smallNumber
       << " "
10      << largeNumber << " " << character << " " << name
       << endl;
```

Output:

```
26 0 3.14 123457 a Ada
```

Esempio: dichiarazioni di variabili e assegnamenti di valori (2/2)

- Il valore booleano `false` viene visualizzato come `0`
- Il numero `123456.789` viene visualizzato come `123457`: per impostazione predefinita, `cout` visualizza al più sei cifre significative
- Il valore memorizzato nella variabile non cambia, e il numero di cifre visualizzate può essere scelto, come vediamo ora

Riepilogo

Tipi di dato:

- `int`, `bool`, `float` e `double`, `char`; le stringhe sono di tipo `string`

Variabili:

- Una variabile memorizza un valore del suo tipo di dato; "=" assegna un nuovo valore e sovrascrive il precedente
- Le variabili devono essere dichiarate, con il loro tipo di dato, prima dell'uso
- `cout` visualizza `false` come `0` e al più sei cifre significative

Flusso di ingresso e uscita

Introduzione: flusso di ingresso e uscita

- Per visualizzare sullo schermo i valori delle variabili, un programma usa la libreria del **flusso di ingresso e uscita**
- Vediamo come includerla, come stampare testo e valori, e come scegliere il numero di cifre visualizzate
- Stampare è il modo in cui vediamo i risultati dei nostri algoritmi

La libreria del flusso di ingresso e uscita

- L'istruzione di inclusione "`#include <iostream>`" permette di usare in C++ la libreria del flusso di ingresso e uscita

Esempio: inclusione

```
1 #include <iostream>
```

Il flusso di uscita

- I valori delle variabili possono essere visualizzati come testo in uscita sullo schermo usando il flusso di uscita "`std::cout`"
- L'operatore di inserimento "<<" serve a inviare i dati al flusso di uscita
- In una sola istruzione `cout` si possono visualizzare più valori concatenandoli con l'operatore "<<"
- Il testo va racchiuso fra virgolette doppie "`"`", mentre i valori delle variabili si scrivono direttamente
- Il manipolatore "`std::endl`" si usa con `std::cout` per andare a capo sullo schermo e svuotare il buffer di uscita

Esempio: stampe sullo schermo

```
1 int n = 7;
2 std::cout << "Il valore della variabile è " << n << std
      ::endl;
```

Output:

```
Il valore della variabile è 7
```

Namespace e numero di cifre decimali

- L'istruzione "`using namespace std;`" semplifica l'uso delle funzioni della libreria standard, eliminando la necessità di anteporre `std::`
- D'ora in poi la usiamo sempre, e scriviamo `cout` e `endl`
- L'istruzione di inclusione "`#include <iomanip>`" fornisce i manipolatori "`fixed`" e "`setprecision(k)`"
- Quando sono scritti insieme in un'istruzione `cout`, visualizzano i numeri in virgola mobile che seguono con esattamente k cifre dopo la virgola decimale

Riepilogo

- `#include <iostream>` fornisce la libreria del flusso di ingresso e uscita
- `cout` visualizza testo (fra virgolette doppie) e valori, concatenati con "<<"; `endl` va a capo
- `using namespace std;` evita il prefisso `std::`
- Con `#include <iomanip>`, `fixed` e `setprecision(k)` visualizzano k cifre dopo la virgola decimale

Operatori

Introduzione: operatori

- Gli operatori sono simboli che eseguono operazioni su variabili e valori
- In base alla loro funzione si classificano in cinque categorie principali:
  - operatori aritmetici
  - operatori di assegnamento
  - operatori di incremento e decremento
  - operatori relazionali
  - operatori logici
- Per ogni categoria vediamo l'elenco degli operatori e un esempio con il suo output

Operatori aritmetici

Operatori aritmetici, usati per eseguire le operazioni matematiche di base:

- + (addizione) $\rightsquigarrow$ somma due valori.
- - (sottrazione) $\rightsquigarrow$ sottrae un valore da un altro.
- * (moltiplicazione) $\rightsquigarrow$ moltiplica due valori.
- / (divisione) $\rightsquigarrow$ divide un valore per un altro.
- % (modulo) $\rightsquigarrow$ restituisce il resto di una divisione.

Esempio: operatori aritmetici

```cpp
int a = 10, b = 3;

cout << "Addizione: " << (a + b) << endl;
cout << "Sottrazione: " << (a - b) << endl;
cout << "Moltiplicazione: " << (a * b) << endl;
cout << "Divisione: " << (a / b) << endl; // Divisione intera
cout << "Resto: " << (a % b) << endl; // Resto della divisione
```

Output:

```
Addizione: 13
Sottrazione: 7
Moltiplicazione: 30
Divisione: 3
Resto: 1
```

Divisione intera e conversione di tipo

- L'operatore di divisione `/` esegue la divisione intera quando è usato con numeri interi: il risultato di `10 / 3` è `3`
- Per ottenere un risultato in virgola mobile, almeno un operando deve essere di tipo `float` o `double`
- Ad esempio, `10.0 / 3` restituisce `3.33333` invece di `3`
- Un valore di tipo `int` viene convertito in un valore di tipo `double` scrivendo "`(double)`" davanti ad esso (**conversione di tipo**)
- Ad esempio, se `a` e `b` sono variabili di tipo `int`, l'espressione `(double)a / b` esegue la divisione in virgola mobile

Esempio: divisione intera e divisione in virgola mobile

```
1 int a = 10, b = 3;
2
3 cout << a / b << endl;           // Divisione intera: 3
4 cout << 10.0 / 3 << endl;        // Divisione in virgola
      mobile
5 cout << (double)a / b << endl;   // Conversione di a in
     double
6 cout << fixed << setprecision(2) << (double)a / b <<
     endl; // Due cifre decimali
```

Output (l'ultima riga richiede `#include <iomanip>`):

```
3
3.33333
3.33333
3.33
```

Operatori di assegnamento

Operatori di assegnamento, usati per assegnare valori alle variabili:

- = (assegnamento) $\rightsquigarrow$ assegna un valore a una variabile.
- += (assegnamento con addizione) $\rightsquigarrow$ somma un valore a una variabile e assegna il risultato.
- -= (assegnamento con sottrazione) $\rightsquigarrow$ sottrae un valore da una variabile e assegna il risultato.
- *= (assegnamento con moltiplicazione) $\rightsquigarrow$ moltiplica una variabile per un valore e assegna il risultato.
- /= (assegnamento con divisione) $\rightsquigarrow$ divide una variabile per un valore e assegna il risultato.
- %= (assegnamento con modulo) $\rightsquigarrow$ applica l'operazione di modulo e assegna il risultato.

Esempio: operatori di assegnamento (1/2)

```
1  int x = 10;  // Assegnamento
2  cout << "Valore iniziale: " << x << endl;
3
4  x += 5;  // Equivale a: x = x + 5
5  cout << "Dopo += 5: " << x << endl;
6
7  x -= 3;  // Equivale a: x = x - 3
8  cout << "Dopo -= 3: " << x << endl;
9
10 x *= 2;  // Equivale a: x = x * 2
11 cout << "Dopo *= 2: " << x << endl;
12
13 x /= 4;  // Equivale a: x = x / 4
14 cout << "Dopo /= 4: " << x << endl;
15
16 x %= 3;  // Equivale a: x = x % 3
17 cout << "Dopo %= 3: " << x << endl;
```

Esempio: operatori di assegnamento (2/2)

Output:

```
Valore iniziale: 10
Dopo += 5: 15
Dopo -= 3: 12
Dopo *= 2: 24
Dopo /= 4: 6
Dopo %= 3: 0
```

Operatori di incremento e decremento

Operatori di incremento e decremento:

- ++ (incremento) $\rightsquigarrow$ aumenta di 1 il valore di una variabile.
- -- (decremento) $\rightsquigarrow$ diminuisce di 1 il valore di una variabile.

L'operatore di incremento ++ si può usare in due forme:

- pre-incremento ("++x"): incrementa il valore prima di restituirlo;
- post-incremento ("x++"): restituisce prima il valore, poi lo incrementa.

La stessa logica vale per l'operatore di decremento (--x e x--).

Esempio: operatori di incremento e decremento (1/2)

```
1  int x = 5;
2
3  cout << "Valore iniziale: " << x << endl; // Stampa 5
4
5  cout << "Pre-incremento (++x): " << ++x << endl; //
       Stampa 6
6
7  cout << "Post-incremento (x++): " << x++ << endl; //
       Stampa 6
8  cout << "Valore dopo il post-incremento: " << x << endl
       ; // Stampa 7
9
10 cout << "Pre-decremento (--x): " << --x << endl; //
       Stampa 6
11
12 cout << "Post-decremento (x--): " << x-- << endl; //
       Stampa 6
13 cout << "Valore dopo il post-decremento: " << x << endl
       ; // Stampa 5
```

Esempio: operatori di incremento e decremento (2/2)

Output:

```
Valore iniziale: 5
Pre-incremento (++x): 6
Post-incremento (x++): 6
Valore dopo il post-incremento: 7
Pre-decremento (--x): 6
Post-decremento (x--): 6
Valore dopo il post-decremento: 5
```

Operatori relazionali

Operatori relazionali, usati per confrontare valori e restituire un risultato booleano:

- == (uguale a) $\rightsquigarrow$ verifica se due valori sono uguali.
- != (diverso da) $\rightsquigarrow$ verifica se due valori sono diversi.
- < (minore di) $\rightsquigarrow$ verifica se un valore è minore di un altro.
- > (maggiore di) $\rightsquigarrow$ verifica se un valore è maggiore di un altro.
- <= (minore o uguale a) $\rightsquigarrow$ verifica se un valore è minore o uguale a un altro.
- >= (maggiore o uguale a) $\rightsquigarrow$ verifica se un valore è maggiore o uguale a un altro.

Risultati booleani

Gli operatori relazionali restituiscono un risultato booleano:

- `true` (visualizzato come `1`) se la condizione è soddisfatta;
- `false` (visualizzato come `0`) se la condizione non è soddisfatta.

Questi operatori sono usati comunemente nelle **istruzioni condizionali** e nei **cicli**.

Esempio: operatori relazionali (1/2)

```
1 int a = 10, b = 5;
2
3 cout << "a == b: " << (a == b) << endl;   // Uguale a
4 cout << "a != b: " << (a != b) << endl;   // Diverso da
5 cout << "a < b: " << (a < b) << endl;     // Minore di
6 cout << "a > b: " << (a > b) << endl;     // Maggiore di
7 cout << "a <= b: " << (a <= b) << endl;   // Minore o
      uguale a
8 cout << "a >= b: " << (a >= b) << endl;   // Maggiore o
      uguale a
```

Esempio: operatori relazionali (2/2)

Output:

```
a == b: 0
a != b: 1
a < b: 0
a > b: 1
a <= b: 0
a >= b: 1
```

Operatori logici

Operatori logici, usati per le operazioni logiche, spesso nelle istruzioni condizionali:

- `&&` (AND logico) $\rightsquigarrow$ restituisce `true` se entrambe le condizioni sono vere.
- `||` (OR logico) $\rightsquigarrow$ restituisce `true` se almeno una condizione è vera.
- `!` (NOT logico) $\rightsquigarrow$ inverte il valore booleano.

Come gli operatori relazionali, gli operatori logici restituiscono valori booleani, `true` (1) o `false` (0), e sono usati comunemente nelle **istruzioni condizionali** e nei **cicli**.

Esempio: operatori logici (1/2)

```
1  bool condition1 = true;
2  bool condition2 = false;
3
4  cout << "condition1: " << condition1 << endl;
5  cout << "condition2: " << condition2 << endl;
6
7  // AND logico: vero solo se entrambe le condizioni sono
       vere
8  cout << "condition1 && condition2: " << (condition1 &&
       condition2) << endl;
9
10 // OR logico: vero se almeno una condizione è vera
11 cout << "condition1 || condition2: " << (condition1 ||
       condition2) << endl;
12
13 // NOT logico: inverte il valore booleano
14 cout << "!condition1: " << !condition1 << endl;
15 cout << "!condition2: " << !condition2 << endl;
```

Esempio: operatori logici (2/2)

Output:

```
condition1: 1
condition2: 0
condition1 && condition2: 0
condition1 || condition2: 1
!condition1: 0
!condition2: 1
```

Riepilogo (1/2)

Operatori aritmetici:

- `+`, `-`, `*`, `/`, `%`
- Fra due valori `int`, `/` è la divisione intera; `(double)` converte un `int` in un `double`

Operatori di assegnamento, di incremento e decremento:

- `x += 5` equivale a `x = x + 5`, e allo stesso modo `-=`, `*=`, `/=`, `%=`
- `++x` incrementa e poi restituisce il valore, `x++` restituisce il valore e poi lo incrementa

Riepilogo (2/2)

Operatori relazionali:

- ==, !=, <, >, <=, >= confrontano due valori

Operatori logici:

- &&, ||, ! combinano valori booleani

Risultati booleani:

- Entrambi restituiscono `true` (visualizzato come `1`) o `false` (visualizzato come `0`), e sono usati nelle istruzioni condizionali e nei cicli

Istruzioni condizionali

Introduzione: istruzioni condizionali

- Gli operatori relazionali e logici producono le condizioni che decidono quali istruzioni vengono eseguite
- Vediamo l'istruzione `if`, con la sua parte `else`, e l'operatore ternario `?:`, la sua forma compatta
- Ogni algoritmo fa delle scelte: le istruzioni condizionali sono il modo in cui C++ le scrive

L'istruzione `if`

- Le istruzioni condizionali permettono di eseguire blocchi di codice diversi in base a condizioni specifiche
- L'istruzione `if` è la struttura di controllo fondamentale di questo tipo
- L'istruzione `if` valuta una condizione scritta fra parentesi tonde `()`
- Se la condizione è **vera**, viene eseguito il blocco di codice fra le parentesi graffe `{}`
- Se la condizione è **falsa**, il programma salta il blocco ed esegue il blocco di codice fra le parentesi graffe `{}` della parte `else`, se presente

Esempio: istruzione condizionale

```cpp
int number = 10;

// Controlla se il numero è positivo
if (number > 0)
{
    cout << "Il numero è positivo." << endl;
}
else
{
    cout << "Il numero non è positivo." << endl;
}
```

Output:

```
Il numero è positivo.
```

Operatore ternario (operatore condizionale)

- L'**operatore ternario** `?:` offre un modo compatto per scrivere semplici istruzioni `if-else`
- È l'unico operatore di C++ che ha tre operandi
- Sintassi: `condizione ? valore_se_vera : valore_se_falsa`
- Se la condizione è **vera**, l'espressione vale `valore_se_vera`; altrimenti vale `valore_se_falsa`

Esempio: operatore ternario (1/2)

```
1  int number = 10;
2
3  // Con l'istruzione if-else
4  string result1;
5  if (number > 0)
6      result1 = "positivo";
7  else
8      result1 = "non positivo";
9
10 cout << "Il numero è " << result1 << endl;
11
12 // Con l'operatore ternario (forma compatta)
13 string result2 = (number > 0) ? "positivo" : "non
       positivo";
14
15 cout << "Il numero è " << result2 << endl;
```

Esempio: operatore ternario (2/2)

Output:

```
Il numero è positivo
Il numero è positivo
```

Esempio: calcolo del massimo fra due numeri (1/2)

```
1  int a = 15, b = 20;
2
3  // Con l'istruzione if-else
4  int max1;
5  if (a > b)
6      max1 = a;
7  else
8      max1 = b;
9
10 cout << "Massimo (if-else): " << max1 << endl;
11
12 // Con l'operatore ternario (forma compatta)
13 int max2 = (a > b) ? a : b;
14
15 cout << "Massimo (ternario): " << max2 << endl;
```

Esempio: calcolo del massimo fra due numeri (2/2)

Output:

```
Massimo (if-else): 20
Massimo (ternario): 20
```

Esempio: verifica se un numero è pari o dispari

```cpp
int n = 7;

// Con l'operatore ternario
string parity = (n % 2 == 0) ? "pari" : "dispari";

cout << n << " è " << parity << endl;
```

Output:

```
7 è dispari
```

Esempio: assegnamento del valore assoluto

```
1 int x = -5;
2
3 // Con l'operatore ternario per calcolare il valore
      assoluto
4 int absValue = (x >= 0) ? x : -x;
5
6 cout << "Il valore assoluto di " << x << " è " <<
      absValue << endl;
```

Output:

```
Il valore assoluto di -5 è 5
```

Esempio: operatori ternari annidati (1/2)

```
1 int score = 85;
2
3 // Operatori ternari annidati per assegnare un voto
4 string grade = (score >= 90) ? "A" :
5                (score >= 80) ? "B" :
6                (score >= 70) ? "C" :
7                (score >= 60) ? "D" : "F";
8
9 cout << "Punteggio: " << score << ", Voto: " << grade
     << endl;
```

Output:

```
Punteggio: 85, Voto: B
```

Esempio: operatori ternari annidati (2/2)

- Le condizioni sono valutate dall'alto verso il basso
- `score >= 90` è falsa e `score >= 80` è vera, quindi il voto è `"B"`
- Per la leggibilità, le condizioni complesse è meglio gestirle con istruzioni `if-else`

Riepilogo

L'istruzione `if`:

- `if (condizione) {...}` esegue il blocco quando la condizione è vera
- Il blocco della parte `else`, se presente, viene eseguito quando la condizione è falsa

L'operatore ternario:

- `condizione ? valore_se_vera : valore_se_falsa` è la forma compatta di un semplice `if-else`
- Utile per il massimo fra due numeri, la parità, il valore assoluto
- Gli operatori ternari annidati sono valutati dall'alto verso il basso; le condizioni complesse è meglio scriverle con `if-else`

Cicli

Introduzione: cicli

- Le istruzioni condizionali scelgono *se* un blocco di codice viene eseguito; i cicli decidono *quante volte* viene eseguito
- Vediamo il ciclo `while`, il ciclo `for`, e le istruzioni `break` e `continue`
- Quasi ogni algoritmo del corso ripete alcune istruzioni: i cicli sono il suo strumento di base

Cicli

- I cicli servono a eseguire ripetutamente un blocco di codice finché una condizione è soddisfatta
- Sono essenziali per automatizzare i compiti ripetitivi e ridurre il codice ridondante
- In C++, i cicli più usati sono il ciclo `while` e il ciclo `for`

Il ciclo `while`

- Il ciclo `while` esegue un blocco di codice finché la condizione fra parentesi tonde `()` vale **vero**
- Quando la condizione è falsa, il ciclo termina

Esempio: ciclo `while`

```
1 int i = 1; // Inizializzazione
2
3 while (i <= 5) // Condizione
4 {
5     cout << "Numero: " << i << endl;
6     i++; // Incremento
7 }
```

Esempio: ciclo `while`

Output:

```
Numero: 1
Numero: 2
Numero: 3
Numero: 4
Numero: 5
```

La condizione `i <= 5` viene valutata sei volte: cinque volte è vera, e la sesta volta (`i` uguale a 6) è falsa e il ciclo termina.

Il ciclo `for`

- Il ciclo `for` è una struttura di ciclo compatta che combina in un'unica istruzione inizializzazione, condizione e incremento (o decremento)

Esempio: ciclo `for`

```
1 for (int i = 1; i <= 5; i++) // Inizializzazione,
      condizione, incremento
2 {
3     cout << "Numero: " << i << endl;
4 }
```

L'output è lo stesso del ciclo `while` dell'esempio precedente.

Le istruzioni `break` e `continue`

- L'istruzione `break` termina immediatamente il ciclo
- L'istruzione `continue` salta il resto dell'iterazione corrente e passa alla successiva

Esempio: break e continue (1/2)

```
1  for (int i = 1; i <= 10; i++)
2  {
3      if (i % 2 == 0)
4      {
5          continue; // Salta i numeri pari
6      }
7      if (i > 7)
8      {
9          break; // Interrompe il ciclo al primo numero
       dispari maggiore di 7
10     }
11     cout << i << " ";
12 }
13 cout << endl;
```

Esempio: `break` e `continue` (2/2)

Output:

```
1 3 5 7
```

- I valori pari di `i` sono saltati da `continue`
- Quando `i` è uguale a 9 l'istruzione `break` termina il ciclo, e il valore 10 non viene mai raggiunto

Riepilogo

Cicli:

- `while (condizione) {...}` ripete il blocco finché la condizione è vera
- `for (inizializzazione; condizione; incremento) {...}` mette le tre parti in un'unica istruzione

Interrompere un ciclo:

- `break` termina immediatamente il ciclo
- `continue` salta il resto dell'iterazione corrente e passa alla successiva

Array

Introduzione: array

- I cicli diventano davvero utili quando scorrono collezioni di valori
- Una collezione di variabili dello stesso tipo di dato è memorizzata in un **array**
- Vediamo gli array di dimensione fissa, i cui valori sono noti quando si scrive il programma, e gli array di dimensione dinamica, la cui dimensione viene decisa durante l'esecuzione
- L'input della maggior parte degli algoritmi del corso è un array

Array di dimensione fissa

- Un array i cui valori sono noti quando si scrive il programma può essere dichiarato elencando i suoi valori fra parentesi graffe
- La dimensione dell'array è il numero di valori elencati, e non può cambiare durante l'esecuzione
- Ogni elemento di un array è identificato dal suo **indice**, che parte da zero, scritto fra parentesi quadre "`[]`"
- Gli elementi di un array di dimensione n sono `b[0]`, `b[1]`, ..., `b[n-1]`

Esempio: array di dimensione fissa

```
1  int b[] = {9, 12, 3, 16}; // Array di 4 interi
2  int n = 4;
3
4  b[2] = 5; // Assegna un nuovo valore al terzo elemento
5
6  for (int i = 0; i < n; i++)
7  {
8      cout << b[i] << " ";
9  }
10 cout << endl;
```

Output:

```
9 12 5 16
```

Array bidimensionali di dimensione fissa

- Un array bidimensionale di dimensione fissa si dichiara con due dimensioni, il numero di righe e il numero di colonne
- I suoi valori si elencano riga per riga
- `int c[2][3] = {{3, 1, 4}, {1, 5, 9}};` dichiara un array con 2 righe e 3 colonne
- `c[1][2]` è l'elemento della seconda riga e terza colonna, uguale a 9

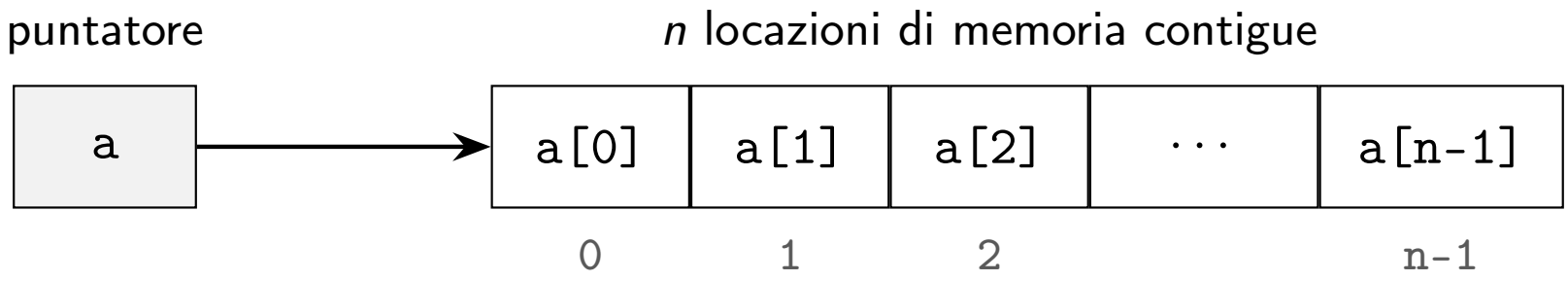
Array di dimensione dinamica (1/2)

- Un **array di dimensione dinamica** è una collezione di variabili dello stesso tipo di dato, memorizzate in locazioni di memoria contigue
- La dimensione dell'array viene determinata durante l'esecuzione dell'algoritmo
- Il simbolo riservato "`*`" si usa per dichiarare un **puntatore**, che memorizza l'indirizzo di memoria del primo elemento di un array di dimensione dinamica
- L'operatore `new` si usa per allocare la memoria di un array di dimensione dinamica, e deve specificare il tipo di dato degli elementi insieme alla dimensione dell'array fra parentesi quadre "`[]`"

Array di dimensione dinamica (2/2)

- Ogni elemento di un array di dimensione dinamica è identificato dal suo indice, che parte da zero, scritto fra parentesi quadre "`[]`"
- Un array di dimensione dinamica richiede la deallocazione della memoria con l'operatore `delete[]`, per liberare la memoria ed evitare perdite di memoria (*memory leak*)

Un array di dimensione dinamica in memoria



Un array di dimensione dinamica n: il puntatore a memorizza l'indirizzo del primo elemento, e gli elementi sono indicizzati da 0 a $n-1$.

Esempio: array di dimensione dinamica (1/2)

```
1  int n = 5; // Dimensione dell'array
2
3  int* a = new int[n]; // Allocazione di un array di
       dimensione n
4
5  for (int i = 0; i < n; i++)
6  {
7      a[i] = i * 2; // Esempio di assegnamento di valori
8  }
9
10 for (int i = 0; i < n; i++)
11 {
12     cout << a[i] << " ";
13 }
14 cout << endl;
15
16 delete[] a;  // Libera la memoria allocata
```

Esempio: array di dimensione dinamica (2/2)

Output:

```
0 2 4 6 8
```

Array bidimensionali di dimensione dinamica (1/2)

- Gli array di dimensione dinamica possono essere multidimensionali
- Ad esempio, il simbolo riservato "**" si usa per dichiarare un puntatore a puntatore, che memorizza l'indirizzo di memoria del primo elemento di un array bidimensionale di dimensione dinamica
- È un array di m puntatori, ciascuno dei quali punta a una riga di n elementi

Array bidimensionali di dimensione dinamica (2/2)

- L'operatore `new` si usa anche per allocare la memoria degli array bidimensionali di dimensione dinamica: prima l'array degli m puntatori alle righe, poi ogni riga
- Ogni elemento di un array bidimensionale di dimensione dinamica è identificato da due indici, entrambi a partire da zero, scritti fra parentesi quadre "`[] []`"
- La memoria viene liberata in ordine inverso: prima ogni riga, poi l'array dei puntatori alle righe

Esempio: array bidimensionali di dimensione dinamica (1/3)

Allocazione della memoria:

```
1 int m = 3; // Numero di righe
2 int n = 4; // Numero di colonne
3
4 int** matrix = new int*[m]; // Allocazione di una
      matrice di dimensione m per n
5 for (int i = 0; i < m; i++)
6 {
7     matrix[i] = new int[n]; // Alloca ogni riga
8 }
```

Esempio: array bidimensionali di dimensione dinamica (2/3)

Assegnamento dei valori e stampa:

```
10 for (int i = 0; i < m; i++)
11 {
12     for (int j = 0; j < n; j++)
13     {
14         matrix[i][j] = i * j; // Esempio di
       assegnamento di valori
15         cout << matrix[i][j] << " ";
16     }
17     cout << endl;
18 }
```

Esempio: array bidimensionali di dimensione dinamica (3/3)

Deallocazione della memoria:

```
20 for (int i = 0; i < m; i++) // Libera la memoria
       allocata
21 {
22     delete[] matrix[i]; // Libera ogni riga
23 }
24 delete[] matrix; // Libera i puntatori alle righe
```

Output:

```
0 0 0 0
0 1 2 3
0 2 4 6
```

Riepilogo

Array di dimensione fissa:

- Si dichiarano elencandone i valori, come in `int b[] = {9, 12, 3, 16};`
- Elementi `b[0]`, `...`, `b[n-1]`; array bidimensionali `c[i][j]`

Array di dimensione dinamica:

- La dimensione viene decisa durante l'esecuzione; il puntatore memorizza l'indirizzo del primo elemento
- Si allocano con `new` e si liberano con `delete[]`
- Bidimensionali: un array di puntatori alle righe (`**`), ogni riga allocata con `new`, e liberati in ordine inverso

La funzione `main` e le funzioni definite dall'utente

Introduzione: la funzione `main` e le funzioni definite dall'utente

- Come nello pseudocodice, un algoritmo è scritto come una funzione che riceve l'input e restituisce l'output
- Vediamo com'è fatta una funzione, il ruolo della funzione `main`, e come si passano i parametri: per valore o per riferimento
- Ogni algoritmo del corso è implementato come una funzione C++

Funzioni

- Le funzioni permettono di scrivere codice modulare e riutilizzabile raggruppando le istruzioni in un'unica unità
- Una funzione può essere chiamata più volte per eseguire lo stesso compito, riducendo la ridondanza e migliorando l'organizzazione del codice
- Di solito le funzioni restituiscono un valore e accettano più parametri in ingresso
- Quando si chiama una funzione, gli argomenti vengono passati ai parametri

Le parti di una funzione

Una funzione in C++ è costituita da:

1. un **tipo di ritorno**: specifica il tipo del valore restituito dalla funzione (`void` se non viene restituito nulla);
2. un **nome della funzione**: l'identificatore univoco della funzione;
3. una **lista dei parametri** (facoltativa): specifica i valori in ingresso della funzione;
4. un **corpo**: contiene il blocco di codice eseguito quando la funzione viene chiamata;
5. un'**istruzione di ritorno** (facoltativa): serve a restituire un valore se la funzione restituisce un valore.

La funzione `main`

- La funzione `main` è il punto di ingresso di un programma `C++`: viene eseguita per prima quando il programma inizia l'esecuzione
- Ogni programma `C++` deve contenere esattamente una funzione `main`
- Il suo valore di ritorno `0` comunica al sistema operativo che il programma è terminato correttamente
- Una funzione deve essere definita (o dichiarata) prima del punto in cui viene chiamata: per questo motivo le funzioni definite dall'utente si scrivono prima di `main`
- Il codice sorgente `C++` può anche essere compilato ed eseguito online, ad esempio al seguente indirizzo: `https://www.mycompiler.io/it/online-c++-compiler`

Esempio: funzioni (1/2)

```cpp
#include <iostream>

using namespace std;

// Definizione della funzione
int addition(int a, int b)
{
    return a + b; // Restituisce la somma
}

// Definizione della funzione
int multiplication(int a, int b)
{
    return a * b; // Restituisce il prodotto
}
```

Esempio: funzioni (2/2)

```
17 int main()
18 {
19     int sum = addition(5, 10); // Chiamata della
       funzione
20     cout << "Somma: " << sum << endl;
21     int prod = multiplication(5, 10); // Chiamata della
        funzione
22     cout << "Prodotto: " << prod << endl;
23
24     return 0;
25 }
```

Output:

```
Somma: 15
Prodotto: 50
```

Parametri passati per valore e per riferimento

Una funzione restituisce al più un valore.

Quando una funzione deve produrre più di un risultato,
oppure deve modificare una variabile del chiamante,
i suoi parametri vengono passati per riferimento.

Per valore, per riferimento, array

- Per impostazione predefinita, un parametro è passato **per valore**: la funzione riceve una copia dell'argomento, e le modifiche fatte alla copia non influenzano la variabile del chiamante
- Un parametro è passato **per riferimento** scrivendo il simbolo riservato "`&`" dopo il suo tipo di dato (ad esempio `int& x`): il parametro è un altro nome per la variabile del chiamante, e ogni modifica fatta dalla funzione è visibile al chiamante
- Un array è passato a una funzione tramite il puntatore al suo primo elemento (ad esempio `int *a`), insieme alla sua dimensione: la funzione lavora direttamente sugli elementi dell'array del chiamante

Esempio: parametri passati per valore e per riferimento (1/4)

```
1  #include <iostream>
2
3  using namespace std;
4
5  void incrementByValue(int x)
6  {
7      x = x + 1; // Modifica solo la copia locale
8  }
9
10 void incrementByReference(int& x)
11 {
12     x = x + 1; // Modifica la variabile del chiamante
13 }
```

Esempio: parametri passati per valore e per riferimento (2/4)

```
15 void fillWithZeros(int n, int* a)
16 {
17     for (int i = 0; i < n; i++)
18     {
19         a[i] = 0; // Modifica l'array del chiamante
20     }
21 }
```

Esempio: parametri passati per valore e per riferimento (3/4)

```
23 int main()
24 {
25     int k = 5;
26     incrementByValue(k);
27     cout << "Dopo incrementByValue: " << k << endl;
28     incrementByReference(k);
29     cout << "Dopo incrementByReference: " << k << endl;
30 
31     int b[] = {4, 7, 1};
32     fillWithZeros(3, b);
33     cout << "Array: " << b[0] << " " << b[1] << " " <<
       b[2] << endl;
34 
35     return 0;
36 }
```

Esempio: parametri passati per valore e per riferimento (4/4)

Output:

```
Dopo incrementByValue: 5
Dopo incrementByReference: 6
Array: 0 0 0
```

Riepilogo

Funzioni:

- Tipo di ritorno, nome, lista dei parametri, corpo, istruzione di ritorno
- `main` è il punto di ingresso, ce n'è esattamente una, e restituisce `0`; le funzioni definite dall'utente si scrivono prima di `main`

Passaggio dei parametri:

- Per valore: la funzione lavora su una copia
- Per riferimento (`int& x`): le modifiche sono visibili al chiamante
- Un array è passato tramite il puntatore al suo primo elemento, insieme alla sua dimensione

Un algoritmo completo: il valore massimo in un array

Introduzione: un algoritmo completo

- Mettiamo ora insieme tutti i pezzi: una funzione, un ciclo, un'istruzione condizionale e un array di dimensione dinamica
- Prendiamo un algoritmo scritto in pseudocodice e lo traduciamo in C++, facendo attenzione agli indici
- È il percorso che segue ogni algoritmo del corso, dallo pseudocodice a un programma che si esegue

Il problema

Consideriamo il problema di trovare il valore massimo in un array a di $n \geq 1$ numeri interi.

- La slide seguente mostra lo pseudocodice dell'algoritmo `FindMaximum` del capitolo «Pseudocodice»: le componenti sono indicizzate da 1 a n

Lo pseudocodice

Algorithm: FindMaximum

Input : Un array $a[1..n]$ di $n \geq 1$ numeri interi
Output: Il valore massimo nell'array a

```
1 function FindMaximum(a, n)
2 |   max := a[1]
3 |   for i := 2 to n do
4 |   |   if a[i] > max then
5 |   |   |   max ← a[i]
6 |   |   end
7 |   end
8 |   return max
9 end
```

Dallo pseudocodice a C++

- Nell'implementazione in `C++` le componenti sono indicizzate da 0 a $n-1$
- Quindi l'inizializzazione usa `a[0]` e il ciclo parte da `i = 1`
- La funzione assume $n \geq 1$, come fa lo pseudocodice: per $n = 0$ l'elemento `a[0]` non esiste

L'implementazione in C++ (1/3)

```cpp
#include <iostream>

using namespace std;

// Funzione che trova il valore massimo di un array
int ArrayMaxValue(int n, int *a)
{
    int max = a[0]; // Inizializza con il primo elemento

    for (int i = 1; i < n; i++)
    {
        if (a[i] > max)
        {
            max = a[i]; // Aggiorna il massimo
        }
    }

    return max;
}
```

L'implementazione in C++ (2/3)

```
21 int main()
22 {
23     int n = 7; // Dimensione dell'array
24
25     int* a = new int[n]; // Allocazione di un array di
       dimensione n
26
27     int values[] = {9, 12, 3, 16, 19, 11, 12};
28     for (int i = 0; i < n; i++)
29     {
30         a[i] = values[i];
31     }
32
33     int max = ArrayMaxValue(n, a);
34     cout << "Valore massimo: " << max << endl;
35
36     delete[] a; // Libera la memoria allocata
37
38     return 0;
39 }
```

L'implementazione in C++ (3/3)

Output:

```
Valore massimo: 19
```

- Ora seguiamo l'esecuzione passo per passo, sull'istanza del programma e su una seconda istanza

Esempio: esecuzione passo per passo (1/2)

In ogni tabella, la colonna i è l'indice del ciclo C++ e la colonna $i+1$ è l'indice corrispondente dello pseudocodice. La prima riga è l'inizializzazione `max = a[0]`.

Istanza 1: $a = [9, 12, 3, 16, 19, 11, 12]$, $n = 7$

i (C++)	$i+1$ (pseudocodice)	`a[i]`	`a[i] > max?`	`max`
–	–	–	–	9
1	2	12	Sì	12
2	3	3	No	12
3	4	16	Sì	16
4	5	19	Sì	19
5	6	11	No	19
6	7	12	No	19

Return: `max` $= 19$

Esempio: esecuzione passo per passo (2/2)

Istanza 2: $a = [3, 7, 2, 9, 1]$, $n = 5$

i (C++)	$i + 1$ (pseudocodice)	`a[i]`	`a[i] > max?`	`max`
–	–	–	–	3
1	2	7	Sì	7
2	3	2	No	7
3	4	9	Sì	9
4	5	1	No	9

Return: `max` $= 9$

Riepilogo

- L'algoritmo `FindMaximum` usa una funzione, un ciclo, un'istruzione condizionale e un array
- Pseudocodice: $max := a[1]$ e il ciclo da $i := 2$ a n
- C++: `max = a[0]` e il ciclo da `i = 1` finché `i < n`; la componente $a[i + 1]$ dello pseudocodice è `a[i]`
- Entrambi assumono $n \geq 1$; sulle due istanze il massimo è 19 e 9

Operatori bit a bit

Introduzione: operatori bit a bit

- I numeri interi gestiti da un programma sono memorizzati in binario, come descritto nel capitolo «Rappresentazione dei numeri»
- C++ fornisce operatori che agiscono direttamente su queste cifre binarie
- Vediamo i sei operatori bit a bit, uno per uno, e poi alcune applicazioni pratiche

Operatori bit a bit

- Gli **operatori bit a bit** eseguono operazioni sui singoli bit dei valori interi
- Sono essenziali per la programmazione a basso livello, per l'ottimizzazione e per lavorare con le rappresentazioni binarie
- I bit sono numerati da destra, a partire dalla posizione 0 (il bit meno significativo)

Gli operatori bit a bit di C++

Operatori bit a bit in `C++`:

- `&` (AND bit a bit) $\rightsquigarrow$ restituisce 1 solo se entrambi i bit corrispondenti sono 1.
- `|` (OR bit a bit) $\rightsquigarrow$ restituisce 1 se almeno uno dei bit corrispondenti è 1.
- `^` (XOR bit a bit) $\rightsquigarrow$ restituisce 1 se i bit corrispondenti sono diversi.
- `~` (NOT bit a bit) $\rightsquigarrow$ inverte tutti i bit (0 diventa 1, 1 diventa 0).
- `<<` (scorrimento a sinistra) $\rightsquigarrow$ sposta i bit a sinistra di un numero specificato di posizioni.
- `>>` (scorrimento a destra) $\rightsquigarrow$ sposta i bit a destra di un numero specificato di posizioni.

AND bit a bit (&)

- L'operatore AND bit a bit confronta ogni bit di due numeri e restituisce 1 solo se entrambi i bit sono 1
- Tavola di verità dell'AND:

Bit A	Bit B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

Esempio: AND bit a bit

```cpp
int a = 12;  // Binario: 1100
int b = 10;  // Binario: 1010
int result = a & b;  // Binario: 1000 (decimale: 8)

cout << "a & b = " << result << endl;  // Stampa: 8
```

Operazione bit per bit:

					Decimale
a	1	1	0	0	12
b	1	0	1	0	10
a & b	1	0	0	0	8

OR bit a bit (|)

- L'operatore OR bit a bit confronta ogni bit di due numeri e restituisce 1 se almeno uno dei bit è 1
- Tavola di verità dell'OR:

Bit A	Bit B	A | B
0	0	0
0	1	1
1	0	1
1	1	1

Esempio: OR bit a bit

```cpp
int a = 12;  // Binario: 1100
int b = 10;  // Binario: 1010
int result = a | b;  // Binario: 1110 (decimale: 14)

cout << "a | b = " << result << endl;  // Stampa: 14
```

Operazione bit per bit:

					Decimale
a	1	1	0	0	12
b	1	0	1	0	10
a | b	1	1	1	0	14

XOR bit a bit (^)

- L'operatore XOR bit a bit (OR esclusivo) confronta ogni bit di due numeri e restituisce 1 se i bit sono diversi
- Tavola di verità dello XOR:

Bit A	Bit B	A ^ B
0	0	0
0	1	1
1	0	1
1	1	0

Esempio: XOR bit a bit

```cpp
int a = 12;  // Binario: 1100
int b = 10;  // Binario: 1010
int result = a ^ b;  // Binario: 0110 (decimale: 6)

cout << "a ^ b = " << result << endl;  // Stampa: 6
```

Operazione bit per bit:

					Decimale
a	1	1	0	0	12
b	1	0	1	0	10
a ^ b	0	1	1	0	6

NOT bit a bit (~)

- L'operatore NOT bit a bit inverte tutti i bit di un numero (0 diventa 1, 1 diventa 0)
- Questo è un operatore unario (opera su un solo operando)
- Il risultato dipende dal numero di bit usati per rappresentare l'intero
- Nell'esempio usiamo il tipo di dato `unsigned char`, che memorizza i numeri naturali da 0 a 255 con 8 bit
- La conversione `(int)` visualizza il risultato come numero invece che come carattere

Esempio: NOT bit a bit

```cpp
unsigned char a = 12;  // Binario (8 bit): 00001100
unsigned char result = ~a;  // Binario: 11110011 (
    decimale: 243)

cout << "~a = " << (int)result << endl;  // Stampa: 243
```

Operazione bit per bit (usando la rappresentazione a 8 bit):

									Decimale
a	0	0	0	0	1	1	0	0	12
~a	1	1	1	1	0	0	1	1	243

Infatti $243 = 255 - 12$: invertire tutti gli 8 bit di un numero x dà $255 - x$.

Scorrimento a sinistra (<<)

- L'operatore di scorrimento a sinistra sposta tutti i bit a sinistra di un numero specificato di posizioni
- Vengono aggiunti zeri sul lato destro
- Scorrere a sinistra di k posizioni equivale a moltiplicare per 2^k (purché il risultato sia rappresentabile dal tipo di dato)

Esempio: scorrimento a sinistra

```cpp
int a = 5;  // Binario: 0101
int result = a << 2;  // Scorrimento a sinistra di 2
    posizioni: 10100 (decimale: 20)

cout << "a << 2 = " << result << endl;  // Stampa: 20
// Equivale a: 5 * 2^2 = 5 * 4 = 20
```

Operazione bit per bit:

							Decimale	Equivalente
a	0	0	0	1	0	1	5	5
a << 1	0	0	1	0	1	0	10	5×2^1
a << 2	0	1	0	1	0	0	20	5×2^2
a << 3	1	0	1	0	0	0	40	5×2^3

Scorrimento a destra (>>)

- L'operatore di scorrimento a destra sposta tutti i bit a destra di un numero specificato di posizioni; i bit che escono a destra vanno persi
- Per i numeri interi non negativi, vengono aggiunti zeri sul lato sinistro
- Per i numeri interi non negativi, scorrere a destra di k posizioni equivale alla divisione intera per 2^k, cioè a $\lfloor x/2^k \rfloor$

Esempio: scorrimento a destra (1/2)

```
1 int a = 20;  // Binario: 10100
2 int result = a >> 2;  // Scorrimento a destra di 2
      posizioni: 00101 (decimale: 5)
3
4 cout << "a >> 2 = " << result << endl;  // Stampa: 5
5 // Equivale a: 20 / 2^2 = 20 / 4 = 5
```

Esempio: scorrimento a destra (2/2)

Operazione bit per bit:

							Decimale	Equivalente
a	0	1	0	1	0	0	20	20
a >> 1	0	0	1	0	1	0	10	$20/2^1$
a >> 2	0	0	0	1	0	1	5	$20/2^2$
a >> 3	0	0	0	0	1	0	2	$\lfloor 20/2^3 \rfloor$

Nell'ultima riga $20/2^3 = 2.5$, e lo scorrimento a destra dà la parte intera 2: il bit 1 in posizione 0 di a >> 2 va perso.

Applicazioni pratiche

- Gli operatori bit a bit offrono soluzioni brevi e veloci per compiti comuni
- Negli esempi seguenti, una **maschera** è un numero la cui rappresentazione binaria ha il bit 1 solo nelle posizioni su cui vogliamo agire
- La maschera `1 << k` ha il bit 1 solo nella posizione k

Esempio: verifica se un numero è pari o dispari

```
1 int n = 15;
2
3 if (n & 1)
4     cout << n << " è dispari" << endl;
5 else
6     cout << n << " è pari" << endl;
7
8 // Spiegazione: n & 1 controlla il bit meno
      significativo
9 // Se vale 1 il numero è dispari, se vale 0 è pari
```

Output:

```
15 è dispari
```

Esempio: scambio di due numeri senza una variabile temporanea (1/2)

```
1 int a = 5, b = 9;
2
3 cout << "Prima dello scambio: a = " << a << ", b = " <<
      b << endl;
4
5 a = a ^ b;  // ora a contiene a XOR b
6 b = a ^ b;  // b = (a XOR b) XOR b = a
7 a = a ^ b;  // a = (a XOR b) XOR a = b
8
9 cout << "Dopo lo scambio: a = " << a << ", b = " << b
     << endl;
```

Output:

```
Prima dello scambio: a = 5, b = 9
Dopo lo scambio: a = 9, b = 5
```

Esempio: scambio di due numeri senza una variabile temporanea (2/2)

Lo scambio funziona perché
`x ^ x` vale 0 e `x ^ 0` vale `x` per ogni intero `x`,
e lo XOR è commutativo e associativo.

Esempio: verifica se un bit specifico è impostato

```cpp
int n = 12;  // Binario: 1100
int position = 2;  // Controlla il bit in posizione 2 (da destra, a partire da 0)

if (n & (1 << position))
    cout << "Il bit " << position << " vale 1" << endl;
else
    cout << "Il bit " << position << " vale 0" << endl;

// Spiegazione: (1 << position) crea una maschera in cui è acceso solo il bit in posizione position
// n & mask controlla se quel bit di n vale 1
```

Output:

```
Il bit 2 vale 1
```

Esempio: impostazione di un bit specifico a 1

```
1 int n = 12;  // Binario: 1100
2 int position = 1;  // Imposta il bit in posizione 1
3
4 n = n | (1 << position);  // Pone il bit in posizione 1
      a 1
5
6 cout << "Risultato: " << n << endl;  // Stampa: 14 (
    binario: 1110)
```

Esempio: azzeramento di un bit specifico a 0

```
1 int n = 14;  // Binario: 1110
2 int position = 1;  // Azzera il bit in posizione 1
3
4 n = n & ~(1 << position);  // Pone il bit in posizione
      1 a 0
5
6 cout << "Risultato: " << n << endl;  // Stampa: 12 (
      binario: 1100)
```

La maschera `~(1 << position)` ha il bit 0 nella posizione data e il bit 1 in tutte le altre posizioni.

Esempio: inversione di un bit specifico

```
int n = 12;  // Binario: 1100
int position = 0;  // Inverte il bit in posizione 0

n = n ^ (1 << position);  // Inverte il bit in posizione 0

cout << "Risultato: " << n << endl;  // Stampa: 13 (binario: 1101)
```

Riepilogo (1/2)

I sei operatori bit a bit:

- `&` (AND), `|` (OR), `^` (XOR) confrontano i bit corrispondenti di due numeri
- `~` (NOT) inverte tutti i bit; il risultato dipende dal numero di bit del tipo di dato

Scorrimenti:

- `x << k` moltiplica x per 2^k (se il risultato è rappresentabile)
- Per $x \geq 0$, `x >> k` calcola $\lfloor x/2^k \rfloor$: i bit che escono a destra vanno persi

Riepilogo (2/2)

Applicazioni pratiche:

- `n & 1` dice se `n` è dispari
- Tre operazioni XOR scambiano due numeri senza una variabile temporanea
- Con la maschera `1 << k`:
  - `n & mask` controlla il bit in posizione k
  - `n | mask` lo pone a 1
  - `n & ~mask` lo azzera
  - `n ^ mask` lo inverte

Punti chiave del capitolo

Introduzione: punti chiave del capitolo

- Raccogliamo i punti chiave del capitolo
- Sono gli strumenti di C++ che usiamo per implementare gli algoritmi del corso

Riepilogo (1/2)

- Un programma `C++` viene compilato (ad esempio con `g++`) e la sua esecuzione parte dalla funzione `main`; ogni istruzione termina con "`;`"
- Ogni variabile è dichiarata con il suo tipo di dato prima dell'uso; la divisione fra due valori `int` è la divisione intera
- Le istruzioni condizionali (`if`, `else`, `?:`) scelgono quali istruzioni vengono eseguite; i cicli (`while`, `for`) le ripetono

Riepilogo (2/2)

- Gli array di `C++` sono indicizzati da 0 a $n-1$, mentre gli array dello pseudocodice sono indicizzati da 1 a n; un array di dimensione dinamica si alloca con `new` e si libera con `delete[]`
- Una funzione riceve i suoi parametri per valore (una copia) o per riferimento (`&`); un array è passato tramite il puntatore al suo primo elemento, insieme alla sua dimensione
- Gli operatori bit a bit agiscono sulla rappresentazione binaria dei numeri interi: `x << k` moltiplica x per 2^k e, per $x \geq 0$, `x >> k` calcola $\lfloor x/2^k \rfloor$